# Bienvenido

Este concurso es la primera Ronda de la Copa Marco Reus - Farmeando Aura - Categoria Inicial.

## Contenido

En este concurso podrás encontrar problemas 100 % lógicos, que te desafiaran.

## Modalidad

El Contest durara $40$ minutos y sera en formato Blitz.

## Personal

| Nombre | Rol |
|---|---|
| Juki | Admin |
| MarcoReus | Juez |
| MarcoReus | Staff |
| MarcoReus | Problem Setter |

# Puesto de frutas

**Time Limit**: 1 second **Memory Limit**: 256 MB

**Type**: Standard **Mode**: Total

## Description

Reus cerró su restaurante y decidió abrir un puesto de frutas.

Su plato especial es una ensalada de frutas que necesita exactamente $2$ bananas y $1$ manzana.

Actualmente, Reus tiene $X$ bananas y $Y$ manzanas.

Determina cuántas ensaladas de frutas puede preparar Reus con las frutas que tiene disponibles.

## Input

- La primera línea contiene un entero $T$ la cantidad de casos de prueba.
- Cada caso de prueba contiene una línea con dos enteros separados por un espacio: $X$ y $Y$.
- $1 \leq T \leq 100$
- $0 \leq X, Y \leq 100$

## Output

Para cada caso de prueba, imprime en una nueva línea la cantidad máxima de ensaladas de frutas que Reus puede preparar.

| Input sample | Output sample |
| --- | --- |
| 3<br>72 50<br>38 93<br>51 4 | 36<br>19<br>4 |

# Marco, Reus y el tablero (Facil)

**Time Limit**: 2 seconds **Memory Limit**: 256 MB

**Type**: Standard **Mode**: Total

## Description

Marco y Reus jugaron algún juego sobre un tablero cuadriculado de tamaño $n \times n$.

Durante el juego colocaron números en las casillas del tablero siguiendo unas reglas complicadas que desconocemos. Sin embargo, el juego ya terminó y cada casilla del tablero contiene exactamente un número.

Para saber quién ha ganado, necesitan contar la cantidad de **casillas ganadoras**.

Para determinar si una casilla específica es ganadora debes realizar lo siguiente:

1. Calcular la suma de todos los números de las casillas que pertenecen a la misma **columna** (incluyendo la casilla seleccionada).
2. Calcular por separado la suma de todos los números de las casillas que pertenecen a la misma **fila** (incluyendo la casilla seleccionada).

Una casilla se considera **ganadora** si la suma de los números de su columna es **estrictamente mayor** que la suma de los números de su fila.

---

Por ejemplo, supongamos que el juego terminó como se muestra en la imagen.

Entonces la casilla morada es ganadora porque:

La suma de los números de su columna es:

$$8 + 3 + 6 + 7 = 24$$

La suma de los números de su fila es:

$$9 + 5 + 3 + 2 = 19$$

Como:

$$24 > 19$$

la casilla es ganadora.

## Input

La primera línea contiene un entero:$n$donde:$1 \leq n \leq 30$

Las siguientes $n$ líneas contienen $n$ enteros separados por espacios.

El $j$-ésimo número de la $i$-ésima línea representa el número ubicado en la casilla que pertenece a la $j$-ésima columna y la $i$-ésima fila del tablero.

Todos los números del tablero son enteros entre:

$1 \leq número \leq 100$

## Output

Imprime un único número: la cantidad de casillas ganadoras

| Input sample | Output sample |
| --- | --- |
| 1<br>1 | 0 |
| 2<br>1 2<br>3 4 | 2 |
| 4<br>5 7 8 4<br>9 5 3 2<br>1 6 6 4<br>9 5 7 3 | 6 |

# Marco, Reus y el Tablero (Dificil)

**Time Limit**: 1 second **Memory Limit**: 256 MB

**Type**: Standard **Mode**: Total

## Description

Marco y Reus jugaron algún juego sobre un tablero cuadriculado de tamaño $n \times n$.

Durante el juego colocaron números en las casillas del tablero siguiendo unas reglas complicadas que desconocemos. Sin embargo, el juego ya terminó y cada casilla del tablero contiene exactamente un número.

Para saber quién ha ganado, necesitan contar la cantidad de **casillas ganadoras**.

Para determinar si una casilla específica es ganadora debes realizar lo siguiente:

1. Calcular la suma de todos los números de las casillas que pertenecen a la misma **columna** (incluyendo la casilla seleccionada).
2. Calcular por separado la suma de todos los números de las casillas que pertenecen a la misma **fila** (incluyendo la casilla seleccionada).

Una casilla se considera **ganadora** si la suma de los números de su columna es **estrictamente mayor** que la suma de los números de su fila.

---

Por ejemplo, supongamos que el juego terminó como se muestra en la imagen.

Entonces la casilla morada es ganadora porque:

La suma de los números de su columna es:

$$8 + 3 + 6 + 7 = 24$$

La suma de los números de su fila es:

$$9 + 5 + 3 + 2 = 19$$

Como:

$$24 > 19$$

la casilla es ganadora.

## Input

La primera línea contiene un entero:$n$donde:$1 \leq n \leq 10^3$

Las siguientes $n$ líneas contienen $n$ enteros separados por espacios.

El $j$-ésimo número de la $i$-ésima línea representa el número ubicado en la casilla que pertenece a la $j$-ésima columna y la $i$-ésima fila del tablero.

Todos los números del tablero son enteros entre:$1 \leq número \leq 10^9$

## Output

Imprime un único número: $\text{la cantidad de casillas ganadoras}$

| Input sample | Output sample |
| --- | --- |
| 1<br>1 | 0 |
| 2<br>1 2<br>3 4 | 2 |
| 4<br>5 7 8 4<br>9 5 3 2<br>1 6 6 4<br>9 5 7 3 | 6 |

# Valor Maximo

**Time Limit**: 1 second **Memory Limit**: 256 MB

**Type**: Standard **Mode**: Total

## Description

Se te da un entero $a$ que consta de $n$ dígitos. También se te da una secuencia de dígitos $s$ de longitud $m$.

El dígito en la posición $j$ ($1 \leq j \leq m$) de la secuencia $s$ significa que puedes elegir una posición arbitraria $i$ ($1 \leq i \leq n$) en $a$ y reemplazar el dígito que se encuentra en la posición $i$ por $s_j$.

Cada elemento de la secuencia $s$ puede participar como máximo en una operación de reemplazo.

Tu tarea es realizar una secuencia de reemplazos de manera que el número dado $a$ obtenga el valor máximo posible. Puedes no utilizar todos los elementos de $s$.

## Input

La primera línea contiene un entero positivo $a$. Su longitud $n$ es positiva y no supera $10^5$.

La segunda línea contiene una secuencia de dígitos $s$. Su longitud $m$ es positiva y no supera $10^5$.

Los dígitos de la secuencia $s$ están escritos consecutivamente, sin ningún separador.

El número dado $a$ no contiene ceros a la izquierda.

## Output

Imprime el valor máximo que se puede obtener a partir de $a$ después de realizar una serie de reemplazos.

Puedes no utilizar todos los elementos de $s$.

El número impreso no debe contener ceros a la izquierda.

| Input sample | Output sample |
|---|---|
| 1024<br>010 | 1124 |
| 987<br>1234567 | 987 |

# Tripletes

**Time Limit**: 2 seconds **Memory Limit**: 256 MB

**Type**: Standard **Mode**: Total

## Description

Reus tiene $n$ puntos sobre un plano. Todos los puntos son diferentes.

Reus quiere encontrar la cantidad de grupos diferentes de tres puntos $(A, B, C)$ tales que el punto $B$ sea el punto medio del segmento $AC$.

Los grupos de tres puntos se consideran **no ordenados**. Es decir, si el punto $B$ es el punto medio del segmento $AC$, entonces los grupos $(A, B, C)$ y $(C, B, A)$ se consideran el mismo grupo.

## Input

La primera línea contiene un único entero $n$ $(3 \leq n \leq 3000)$ — la cantidad de puntos que tiene Reus.

Las siguientes $n$ líneas contienen las coordenadas de los puntos.

La $i$-ésima línea contiene las coordenadas del $i$-ésimo punto: dos enteros separados por un espacio $x_i, y_i$ $(-1000 \leq x_i, y_i \leq 1000)$.

Se garantiza que todos los puntos dados son diferentes.

## Output

Imprime un único número — la cantidad de grupos diferentes de tres puntos que cumplen la condición.

| Input sample | Output sample |
|---|---|
| 3<br>1 1<br>2 2<br>3 3 | 1 |
| 3<br>0 0<br>-1 0<br>0 1 | 0 |

# Extensiones

**Time Limit**: 2 seconds **Memory Limit**: 256 MB

**Type**: Standard **Mode**: Total

## Description

Reus, un administrador de sistemas, está nuevamente en problemas.

Debido a un error en algún script, una lista con los nombres de archivos muy importantes resultó dañada.

Como se trataba de archivos del sistema de archivos **BerFS**, se sabe que cada nombre de archivo tiene la forma:

`name.ext`

donde:

- **`name`** es una cadena formada por letras latinas minúsculas y su longitud está entre $1$ y $8$ caracteres.
- **`ext`** es una cadena formada por letras latinas minúsculas y su longitud está entre $1$ y $3$ caracteres.

Por ejemplo:

`read.me`
`example.txt`
`b.cpp`

son nombres de archivo válidos.

En cambio:

`version.info`
`ntldr`
`contestdata.zip`

no son nombres de archivo válidos.

El daño en la lista provocó que todos los nombres de archivo fueran escritos uno después de otro, sin ningún separador entre ellos.

Por lo tanto, ahora Reus tiene una única cadena.

Reus necesita arreglar todo lo antes posible. Debe dividir la cadena resultante en varias partes de manera que cada una de ellas sea un nombre de archivo válido en **BerFS**.

Como Reus ya ha demostrado que la programación no es precisamente su fuerte, ayúdalo a reconstruir la lista de archivos.

La lista resultante puede contener nombres de archivos repetidos.

## Input

La entrada contiene una única cadena $s$, cuya longitud está entre:

$1 \leq |s| \leq 4 \cdot 10^5$

La cadena puede contener únicamente:

- letras latinas minúsculas (`a` - `z`);
- puntos (`.`).

### Output

En la primera línea imprime: `YES` si es posible dividir la cadena $s$ en partes de manera que cada una sea un nombre de archivo válido.
En ese caso, las siguientes líneas deben contener los nombres de archivo obtenidos, uno por línea, respetando el mismo orden en el que aparecen dentro de $s$.
La partición resultante puede contener nombres de archivos repetidos.
Si existen varias soluciones posibles, imprime aquella donde la extension sea lo mas corta posible.
Si no es posible realizar una partición válida, imprime `NO`

| Input sample | Output sample |
|---|---|
| read.meexample.txtb.cpp | YES<br>read.m<br>eexample.t<br>xtb.cpp |
| version.infontldrcontestdata.zip | NO |